

Path Traversal

파일 경로와 기준 디렉터리

서버는 특정 디렉터리 안의 파일만 제공하려고 한다

```
/srv/  
├── config.yml          ← ../../config.yml의 도착지  
└── app/  
    ├── config.yml     ← ../config.yml의 도착지  
    └── files/  
        └── report.pdf  ← 출발 위치
```

기준 디렉터리 `/srv/app/files`

허용 파일 `/srv/app/files/report.pdf`

보호 파일 `/srv/config.yml`

사용자에게 허용할 범위는 `/srv/app/files` 내부

Path Traversal이란

허용 디렉터리 밖을 가리키는 경로를 차단하지 못함

```
/srv/  
├── config.yml  
└── app/  
    ├── config.yml  
    └── files/  
        └── report.pdf
```

← ../../config.yml의 도착지
← ../config.yml의 도착지
← 출발 위치

정상 입력 report.pdf

공격 입력 ../../config.yml

결과 의도한 범위 밖의 파일에 접근

정상적인 다운로드 요청

요청에 담긴 파일명이 기준 디렉터리 안으로 연결된다

- ① GET /download?file=report.pdf
- ② base = /srv/app/files
- ③ /srv/app/files/report.pdf
- ④ report.pdf 반환

취약한 다운로드 요청

..는 현재 상위 디렉토리를 뜻한다

① GET /download?file=../../config.yml

② base = /srv/app/files

③ /srv/app/files/../../config.yml

④ /srv/config.yml

=> 허용 범위 밖 파일 접근

상대 경로를 이용한 탈출

부모 디렉터리로 반복해서 이동한다

Unix 계열 `../secret.txt`

Windows `..\secret.txt`

여러 단계 `../../../config.yml`

절대 경로와 Windows UNC

..가 없어도 기준 디렉터리를 벗어날 수 있다

Unix 절대경로	/etc/passwd
-----------	-------------

Windows 드라이브	C:\Windows\win.ini
--------------	--------------------

Windows UNC	//server/share
-------------	----------------

인코딩과 디코딩 시점

검사하는 문자열과 파일 API가 받는 문자열이 같아야 한다

원래 입력	../secret.txt
-------	---------------

URL 인코딩	%2e%2e%2fsecret.txt
---------	---------------------

이중 인코딩	%252e%252e%252fsecret.txt
--------	---------------------------

인코딩과 디코딩 시점

```
base = "/srv/app/files"  
raw_input = "%252e%252e%252fsecret.txt"
```

```
# 웹 프레임워크가 URL을 한 번 디코딩했다고 가정  
checked_value = unquote(raw_input)  
print("검사할 때의 값:", checked_value)
```

```
if "../" in checked_value:  
    print("검사 결과: 차단")  
else:  
    print("검사 결과: 통과")  
  
# 애플리케이션의 다른 코드가 또 디코딩하는 실수  
used_value = unquote(checked_value)  
target = posixpath.normpath(posixpath.join(base, used_value))
```

인코딩과 디코딩 시점

검사할 때의 값: %2e%2e%2fsecret.txt

검사 결과: 통과

파일 경로에 사용할 값: ../secret.txt

최종 파일 경로: /srv/app/secret.txt

경로에 영향을 주는 입력

파일 경로에 쓰이는 외부 값은 모두 확인해야 한다.

URL 파라미터 ?file=report.pdf

업로드 파일명 filename=photo.png

압축 파일 항목 images/a.png

설정 값 template=blue

파일 연산에 따른 영향

경로가 도달하는 파일 연산을 찾아야 한다

읽기	설정.소스.키 파일 노출
쓰기	중요한 파일 생성 또는 덮어쓰기
삭제	파일 손상과 서비스 장애
실행	덮어쓴 파일이 실행될 때 추가 피해

경로 정규화와 실제 경로

문자열을 정리한 결과와 실제 파일 위치는 다를 수 있다

문자열 수준 정리 `/a/b/../c` → `/a/c`

실제 경로 해석 symlink가 가리키는 위치까지 확인

경계 검사 최종 경로가 기준 디렉터리 내부인가?

심볼릭 링크의 영향

디렉터리 안에 보이는 경로가 밖의 파일로 이어질 수 있다

/srv/app/files/link → /srv/private

요청 경로 /srv/app/files/link/key.txt

실제 접근 /srv/private/key.txt

압축 해제와 경로 탈출

압축 파일 안의 경로도 파일 저장 위치를 결정한다

압축 파일 안의 이름: ../outside.txt

압축을 풀 폴더: /work/unpack

합친 경로: /work/unpack/../outside.txt

실제 저장되는 곳: /work/outside.txt

취약한 문자열 필터

위험해 보이는 글자를 지우는 방식은 모든 부분을 막을 수 없다.

```
filename = filename.replace("../", "")
```

빠뜨릴 수 있는 입력

- Windows의 다른 경로 구분자
- /로 시작하는 절대 경로
- 나중에 디코딩되는 값
- 심볼릭 링크가 가리키는 위치

경로를 받아야 할 때의 검사

최종 경로가 기준 디렉터리 안에 있는지 경로 단위로 비교한다

```
from pathlib import Path

BASE = Path("/srv/app/files").resolve()
candidate = (BASE / user_value).resolve()

if candidate != BASE and BASE not in candidate.parents:
    raise ValueError("허용 범위 밖 경로")
```

안전한 설계: 파일 ID 사용

클라이언트는 파일 경로 대신 서버가 발급한 ID를 보낸다

클라이언트 요청 /download?file_id=17

서버 내부 매핑 17 → /srv/app/files/9c8e.pdf

서버 권한 검사 요청자가 17번 파일을 볼 수 있는가?

Path Traversal 분석 기준

실제 취약점에서는 네 가지를 순서대로 확인한다

- ① 입력: 누가 파일명이나 경로를 결정하는가?
- ② 처리: 디코딩·결합·정규화는 어떤 순서인가?
- ③ 연산: 읽기·쓰기·삭제 중 무엇을 하는가?
- ④ 경계: 실제 도착 경로가 허용 범위 안인가?

Path Traversal 1-day

GitPython: CVE-2023-41040

GitPython이란?

Python 프로그램 안에서 Git 저장소를 다루게 해 주는 라이브러리

터미널에서 Git 명령어를 직접 입력하는 대신 Python에서 저장소를 열 수 있다.

```
import git

repo = git.Repo("project") # project 폴더의 Git 저장소 열기
commit = repo.commit("main") # main이 가리키는 기존 커밋 조회
```

GitPython: CVE-2023-41040

- CVE-2023-41040
- CVSS: 4.0(medium)
- 취약점 유형: CWE-22(Path Traversal)
- GitPython 패키지 3.1.36 버전 이하
- 핵심
 - GitPython이 Git의 참조 이름을 파일 경로로 사용하면서,
.git 폴더 밖에 나가는지 허용 범위를 확인하지 않아서 생긴 문제

Git 참조와 .git 폴더

참조 정보는 .git 안에, README.md는 그 밖에 있다

```
project/  
├── .git/  
│   └── refs/  
└── README.md
```

취약 코드: 경로 결합 후 읽기

참조 이름을 확인하지 않은 채 파일 경로로 사용했다

```
with open(os.path.join(repodir, str(ref_path)),  
          "rt", encoding="UTF-8") as fp:  
    value = fp.read().rstrip()
```

repodir .git 디렉터리

ref_path 찾으려는 참조 이름

open() 합쳐진 경로의 파일을 실제로 읽음

../README.md의 경로 탈출

..가 .git의 상위 폴더로 이동시킨다

기준 디렉터리	project/.git
입력	../README.md
합친 경로	project/.git/../README.md
실제 경로	project/README.md

공개된 재현 코드

참조 이름 자리에 ../README.md를 넣는다

```
import git

r = git.Repo(".")

# This will make GitPython read the README.md file from the root of the repo
r.commit("../README.md")
```

git.Repo(".") 현재 위치의 Git 저장소 열기

r.commit(...) README.md를 참조처럼 읽으려 시도

재현의 한계와 영향

직접 확인 파일 내용 반환은 공식 재현에서 입증되지 않음

간접 영향 사용 방식에 따라 파일 존재 여부 추정 가능

가용성 큰 파일을 읽으면 처리 지연·장애 가능

원격 조건 앱이 외부 입력을 참조 이름으로 넘겨야 함

네 가지 분석 질문

- ① 입력은? Repo.commit() 등에 전달된 ref_path
- ② 경로 처리는? .git 디렉터리와 ref_path 결합
- ③ 파일 연산은? open()으로 파일을 읽음
- ④ 경계는? .git 안에서만 읽어야 하지만 밖으로 나감

패치: 파일 열기 전에 거부

ref_path에 ..가 있으면 open()에 도달하지 않는다

```
if ".." in str(ref_path):  
    raise ValueError(f"Invalid reference '{ref_path}'")
```

변경 위치 git/refs/symbolic.py

효과 ../README.md 같은 참조를 조기 거부

수정 버전 GitPython 3.1.37

Werkzeug: CVE-2024-49766

Werkzeug란?

Python 웹 애플리케이션에서 사용하는 도구 모음이다.

대표적으로 `send_from_directory()`와 `safe_join()`이 있다.

`send_from_directory()` 지정한 폴더의 파일을 제공

`safe_join()` 안전한 경로를 반환하고 위험하면 `None` 반환

Werkzeug: CVE-2024-49766

- CVE-2024-49766
- CVSS: 6.3(medium)
- 취약점 유형: CWE-22(Path Traversal)
- Werkzeug 3.0.5 버전 이하 / Windows / Python 3.11 미만
- 핵심
 - Safe_join()에서 os.path.isabs(filename)으로 절대 경로를 검사할 때
 - UNC 네트워크 경로를 제대로 검사하지 못함.

UNC 네트워크 경로

//server/share는 Windows의 공유 폴더를 가리킨다

허용된 폴더	C:/service/public
사용자 입력	//fileserver/shared/secret.txt
원래 기대	공개 폴더 밖이므로 거부
위험한 결과	네트워크 공유 위치를 가리키는 경로 반환

검사 단계: UNC 경로를 놓침

구버전 Windows Python의 `isabs()`가 `False`를 반환

```
os.path.isabs("//server/share") # False
```

절대경로 검사 통과

../ 검사 해당 없음

결과 `safe_join()`의 경로 결합 단계로 진행

경로 결합: 공개 폴더가 사라짐

기준 폴더	C:/service/public
입력	//fileserver/shared/secret.txt
결합	posixpath.join(기준 폴더, 입력)
반환 경로	//fileserver/shared/secret.txt

반환 경로와 실제 파일 읽기

직접 결과	<code>safe_join()</code> 이 공개 폴더 밖 UNC 경로를 반환
후속 연산	<code>send_from_directory()</code> 등이 파일을 읽어 제공
접근 조건	대상 파일 존재와 서버의 접근 권한 필요
핵심 영향	의도하지 않은 데이터 접근 가능성

네 가지 분석 질문

- ① 입력은? 사용자가 요청한 파일 경로
- ② 경로 처리는? `safe_join()`의 검사와 `posixpath.join()`
- ③ 파일 연산은? 파일 제공 기능이 반환 경로를 읽음
- ④ 경계는? 공개 폴더 밖의 UNC 경로

패치: /로 시작하면 거부

```
# ntpath.isabs doesn't catch this on Python < 3.11  
or filename.startswith("/")
```

변경 위치	src/werkzeug/security.py
효과	//server/share 같은 입력 거부
수정 버전	Werkzeug 3.0.6

Keras: CVE-2026-12482

- CVE-2026-12482
- 취약점 유형: CWE-22 (Path Traversal)
- CVSS: 3.1 (Low)
- Keras 3.12.3 미만 / 3.13.0 이상 3.15.0 미만
- 핵심: 검사는 했지만, 검사할 때 계산한 경로와 압축 해제 때 실제로 따라간 경로가 달랐던 경우

Keras에서 압축 해제가 일어나는 곳

`get_file(extract=True)`는 받은 파일을 풀 수 있다

Keras	머신러닝에 사용하는 Python 라이브러리
-------	-------------------------

<code>get_file()</code>	파일을 내려받아 캐시에 저장
-------------------------	-----------------

<code>extract=True</code>	TAR 등의 압축파일을 해제
---------------------------	-----------------

보안 경계	정해진 압축 해제 폴더 안에만 생성
-------	---------------------

공격자가 만든 TAR 파일

두 항목의 순서와 경로가 중요하다

압축 해제 폴더 /work/unpack

① link 심볼릭 링크 → "."

② link/../ESCAPED 일반 파일

공격자의 목표 /work/ESCAPED 생성

link와 ..를 따로 이해하기

link는 바로가기, ..는 그 위치의 상위 폴더

/work/unpack/link → /work/unpack ("." 대상)

link를 따라감 → /work/unpack에 도착

그다음 .. → /work로 이동

따라서 실제 위치 → /work/ESCAPED

패치 전 경로 계산

abspath()를 먼저 호출한 뒤 realpath()를 호출

```
return os.path.realpath(os.path.abspath(path))
```

abspath() .과 ..를 문자열 기준으로 정리

realpath() 존재하는 심볼릭 링크를 따라 실제 경로 계산

문제 단계에서 link가 사라지면 확인 불가

검사할 때 계산한 경로

abspath()가 link/..를 글자 기준으로 지운다

검사 입력 /work/unpack/link/../ESCAPED

abspath() /work/unpack/ESCAPED

realpath() /work/unpack/ESCAPED

검사 결과 unpack 안이므로 허용한다고 판단

압축 해제 때 따라간 경로

실제 파일 생성은 심볼릭 링크를 먼저 따라간다

시작 /work/unpack

link 따라감 /work/unpack (link → ".")

.. 적용 /work (상위 폴더)

파일 생성 /work/ESCAPED ← 허용 폴더 밖

검사 경로 ≠ 실제 저장 경로

보안 검사는 통과했지만 파일은 폴더 밖에 생성

검사가 본 경로 /work/unpack/ESCAPED

실제 저장 경로 /work/ESCAPED

벗어난 경계 /work/unpack

영향 폴더 밖 파일 생성. 덮어쓰기 가능

네 가지 분석 질문

입력, 경로 처리, 파일 연산, 경계를 구분한다

- ① 입력은? 공격자가 만든 TAR 항목의 경로
- ② 경로 처리는? abspath() 후 realpath()로 검사
- ③ 파일 연산은? 압축 해제가 파일을 생성
- ④ 경계는? /work/unpack 밖으로 벗어남

패치: realpath()를 바로 적용

심볼릭 링크를 반영한 경로로 포함 여부 검사

```
return os.path.realpath(path)
```

변경점 앞의 abspath() 제거

계산 결과 /work/ESCAPED로 경로를 확인

후속 검사 /work/unpack 밖이므로 해당 항목 거부